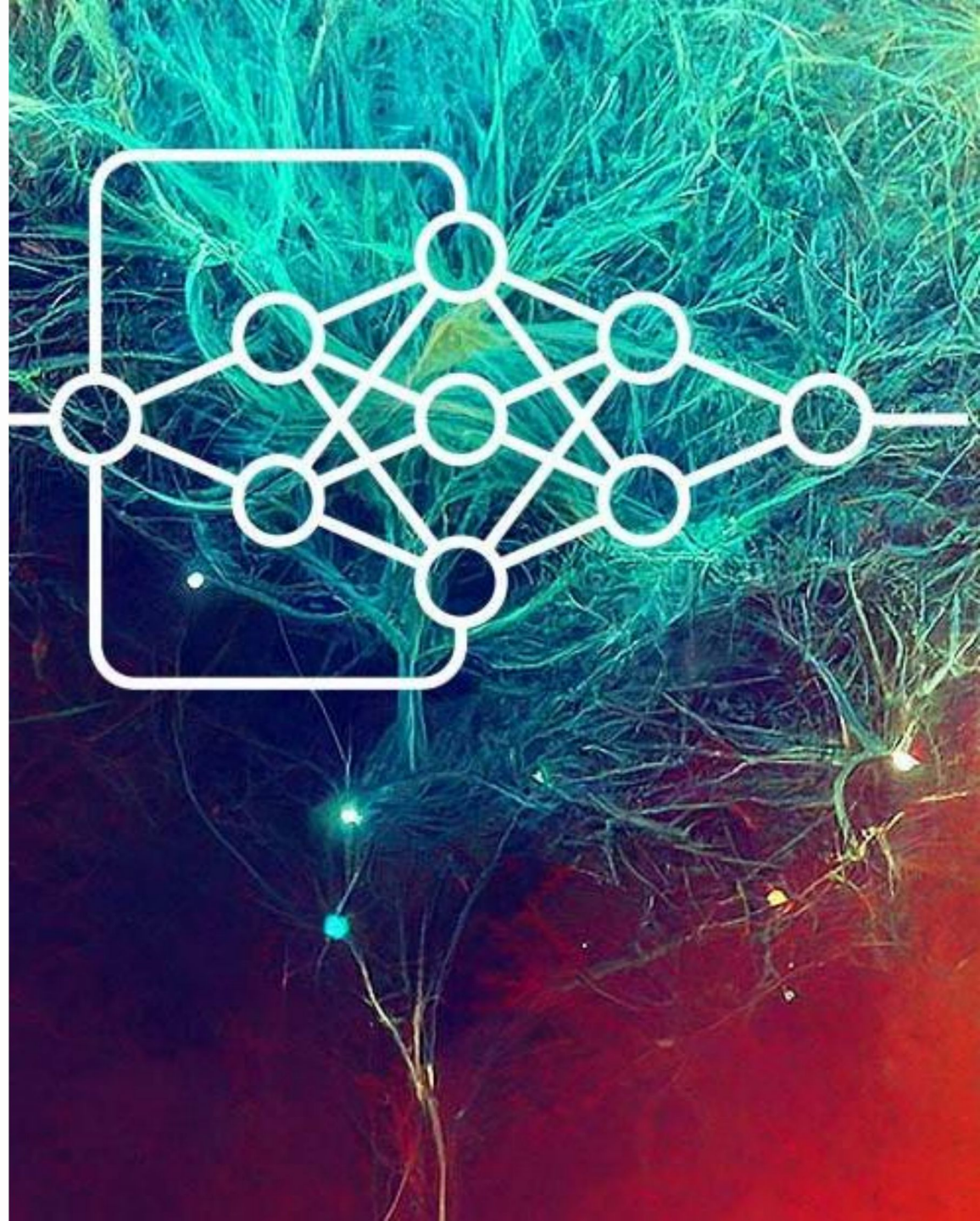


Свёрточные нейронные сети и Evasion-атаки

Математический аппарат CNN, эволюция архитектур и первый класс атак на глубокие модели: от «intriguing properties» к Fast Gradient Sign Method



Цель лекции

Часть I — Математика CNN

Формализация операции двумерной дискретной свёртки, вывод размера feature map, операция max-pooling, понятие рецептивного поля и его рекуррентный рост. Сравнение архитектур от LeNet до ResNet с акцентом на ключевые инженерные решения каждой эпохи.

Часть II — Adversarial ML

Переход к первому классу атак на глубокие модели — evasion-атакам. Разбор основополагающей работы Szegedy et al. (2013) об «intriguing properties» нейронных сетей, линейная гипотеза Гудфеллоу и строгий математический вывод Fast Gradient Sign Method (FGSM).

К концу лекции слушатель понимает, **почему** существуют adversarial examples и как их конструировать формально.

Дорожная карта лекции

01

Математика свёртки

Дискретная 2D-свёртка, страйд, паддинг, пулинг

02

Рецептивное поле

Рекуррентная формула, иерархия признаков, VGG vs AlexNet

03

Эволюция архитектур

LeNet → AlexNet → VGG → ResNet, skip-connections

04

Intriguing properties

Два контринтуитивных свойства, трансферируемость атак

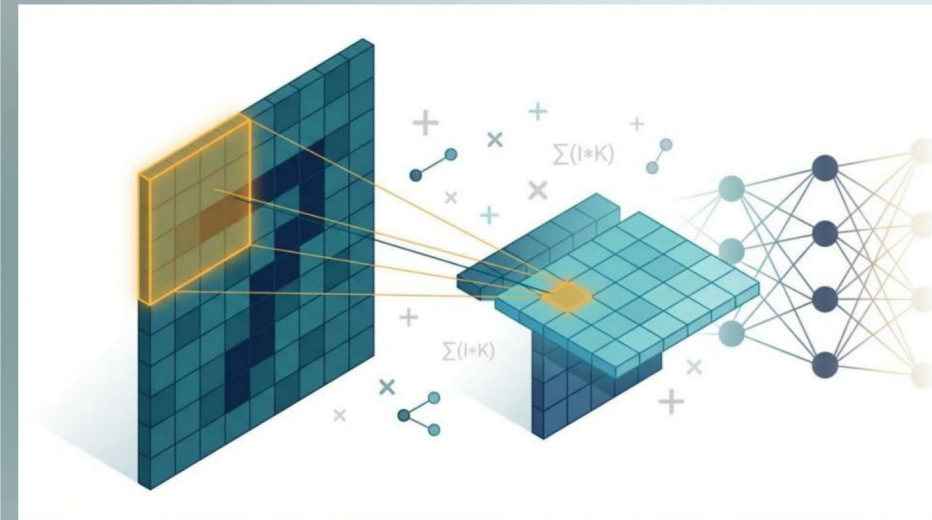
05

Линейная гипотеза и FGSM

Вывод метода, adversarial training, ограничения

1. Операция двумерной дискретной свёртки

Формальная отправная точка всех CNN — дискретная 2D-свёртка входного тензора с обучаемым ядром.



Формальное определение свёртки

Пусть входной тензор $X \in \mathbb{R}^{H \times W \times C}$ (высота, ширина, число каналов) и набор ядер $K \in \mathbb{R}^{k \times k \times C}$ в фильтре F.

Значение выходной карты признаков в позиции (i, j) :

$$Z_{i,j} = \sum_{c=1}^C \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X_{i+m, j+n, c} \cdot K_{m,n,c} + b$$

где b — обучаемое смещение (bias). Тройная сумма пробегает все C каналов и все k^2 позиций ядра, выполняя поэлементное умножение и суммирование (скалярное произведение «распластанного» патча с ядром).

Размер выходной карты признаков

При страйде s и паддинге p :

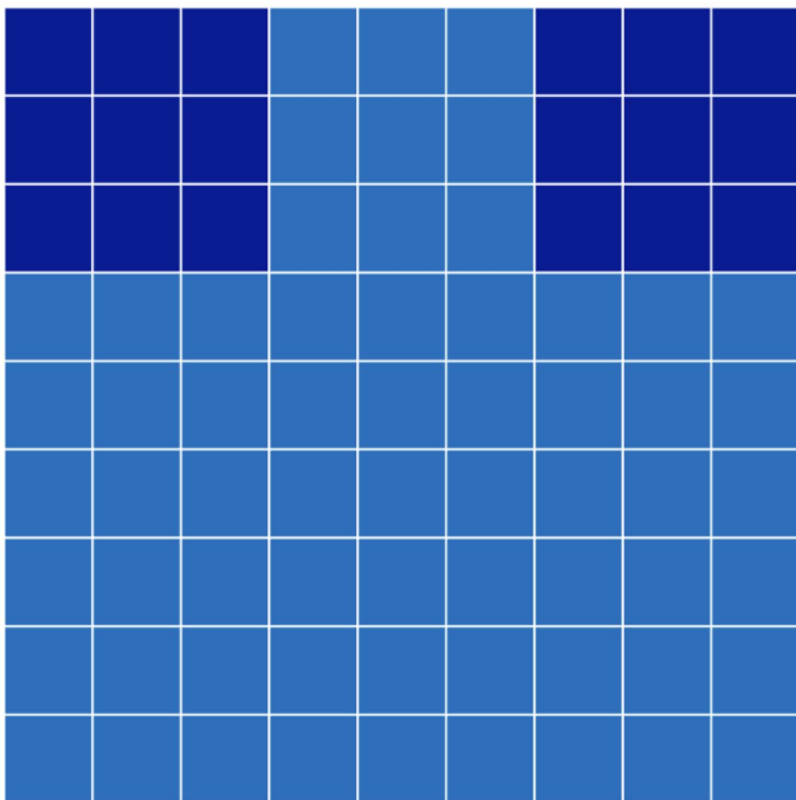
$$H_{out} = \left\lfloor \frac{H - k + 2p}{s} \right\rfloor + 1$$

Аналогично по ширине. Чем больше страйд — тем агрессивнее субдискретизация. Паддинг $p = (k - 1)/2$ при $s = 1$ сохраняет размер («same» padding).

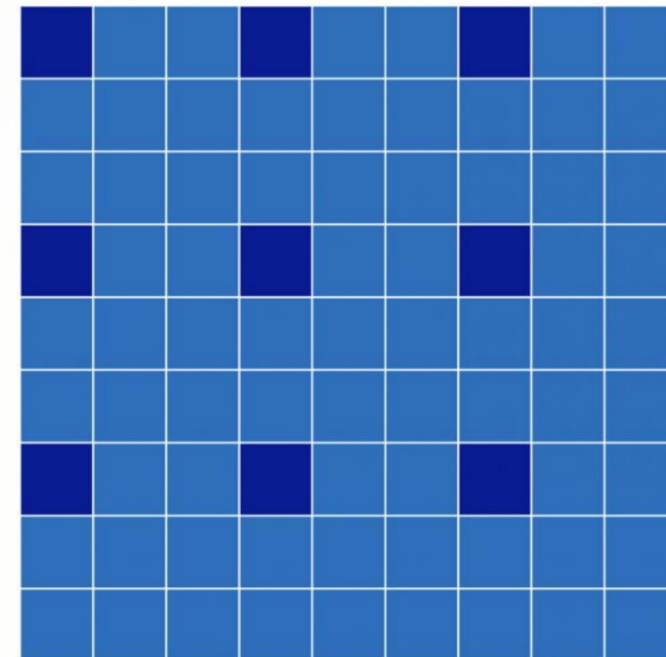
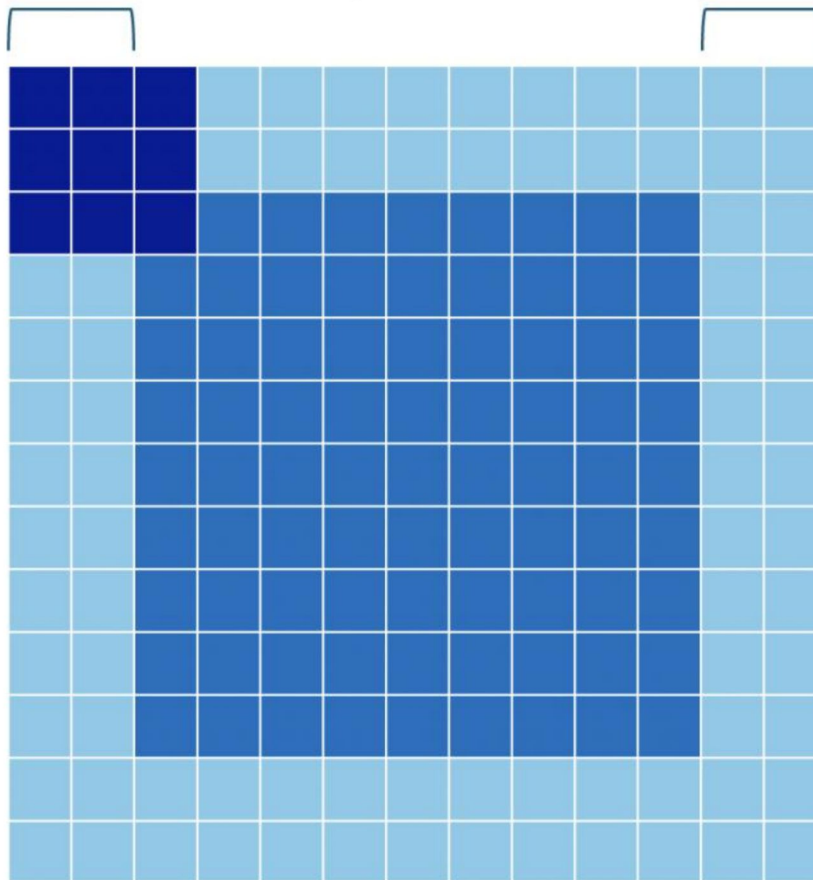
Ключевые свойства

- **Разделяемость весов:** одно ядро применяется ко всем позициям — резкое сокращение параметров.
- **Локальная связность:** нейрон связан только с локальной областью входа.
- **Трансляционная эквивариантность:** сдвиг объекта во входе сдвигает feature map, не меняя активаций.

$s = 6$



$p = 2$



Дилатация равная трем.

Операция пулинга (Max-Pooling)

Max-pooling снижает пространственную размерность карты признаков, выбирая максимум в окне $k \times k$:

$$P_{i,j} = \max_{0 \leq m, n < k} X_{i+m, j+n}$$

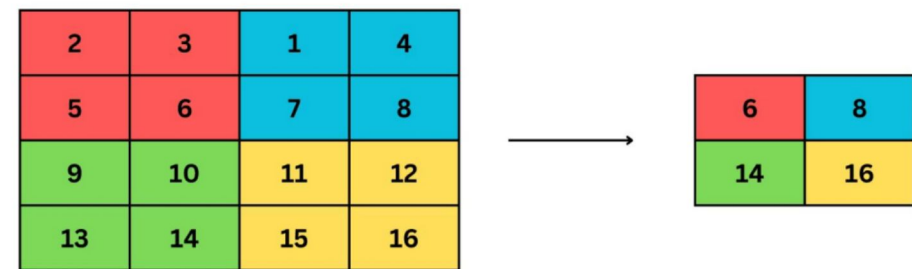
Формула размера выхода та же, что для свёртки. Пулинг **не имеет обучаемых параметров** — это детерминированная операция агрегации.

Зачем нужен пулинг?

- Субдискретизация уменьшает объём вычислений на следующих слоях.
- Достигается **инвариантность к малым трансляциям**: максимальная активация в окне сохраняется при небольших сдвигах признака.
- Неявная регуляризация за счёт потери точной позиционной информации.

❏ В современных архитектурах (ResNet, EfficientNet) пулинг нередко заменяется свёрткой со страйдом $s = 2$ что сохраняет обучаемость параметров агрегации.

Max Pooling



Select the maximum value from each window.

2. Рецептивное поле и иерархия признаков

Для слоя $l = 1, \dots, L$:

- k_l — размер ядра;
- s_l — stride;
- d_l — dilation;
- p_l — padding;
- r_l — размер рецептивного поля после l -го слоя;
- j_l — «jump» / effective stride: расстояние в пикселях исходного изображения между центрами рецептивных полей соседних нейронов карты признаков.

эффективный размер ядра

$$k_l^{\text{eff}} = d_l(k_l - 1) + 1$$

Пример. Ядро 3x3 с dilation $d=2$ покрывает область 5x5, хотя имеет только 9 обучаемых весов

Начинаем от входа:

$$r_0 = 1, \quad j_0 = 1.$$

Пусть нейрон текущего слоя получает значения из окна длины k_l^{eff} на предыдущей карте признаков.

1. Одно значение предыдущей карты соответствует области r_{l-1} на исходном изображении.
2. Между центрами рецептивных полей соседних значений предыдущей карты расстояние равно j_{l-1} .
3. Окно длины k_l^{eff} содержит $k_l^{\text{eff}} - 1$ промежутков.
4. Поэтому добавочная ширина видимой области равна

$$(k_l^{\text{eff}} - 1)j_{l-1}.$$

Отсюда основная рекурсия:

$$r_l = r_{l-1} + [d_l(k_l - 1)] j_{l-1}$$

или, если дилатация отсутствует ($d_l = 1$):

$$r_l = r_{l-1} + (k_l - 1)j_{l-1}.$$

При этом шаг «перескока» обновляется как

$$j_l = j_{l-1} s_l$$

и, следовательно,

$$j_l = \prod_{i=1}^l s_i.$$

2. Рецептивное поле и иерархия признаков

Рецептивное поле (Receptive Field, RF) нейрона — область пикселей входного изображения, оказывающих влияние на значение данного нейрона. Понимание RF критично для выбора глубины архитектуры и размера ядер.

$$RF_L = RF_{L-1} + (k_L - 1) \prod_{i=1}^{L-1} s_i$$

Рекуррентная формула: рецептивное поле после L слоёв с ядрами k_l и страйдами s_l . Произведение страйдов предыдущих слоёв — «масштаб» шага, с которым ядро L -го слоя «видит» входное пространство.

Пример: VGG и «дешёвое» рецептивное поле

Два последовательных слоя 3×3 (страйд 1) дают $RF = 5 \times 5$ — эквивалент одной свёртки 5×5 .

Три слоя $3 \times 3 \equiv RF 7 \times 7$ одного ядра.

Что по параметрам?

Иерархия признаков в CNN

- **Ранние слои** (малое RF): ориентированные края, цветовые переходы, пространственные частоты.
- **Средние слои**: текстуры, части объектов (угол глаза, колесо автомобиля).
- **Глубокие слои** (большое RF): семантически целые концепции — лицо, автомобиль, кошка.

Эта иерархия подтверждается визуализацией feature maps и тепловыми картами Grad-CAM, показывающими, на какие регионы входа «опирается» решение сети.

Сравним число параметров для одного «большого» ядра и стека из слоёв 3×3. Будем считать, что число каналов на входе — C_{in} , на выходе — C_{out} , а промежуточные слои имеют по C каналов.

Одна свёртка 5×5

$$params = 5 \times 5 \times C_{in} \times C_{out} = 25 C_{in} C_{out}$$

Две свёртки 3×3 (RF = 5×5)

$$params = (3 \times 3 \times C_{in} \times C) + (3 \times 3 \times C \times C_{out}) = 9C(C_{in} + C_{out})$$

Если $C_{in} = C_{out} = C$ (типичный случай, как в VGG, где каналы сохраняются):

$$\underbrace{9C^2 + 9C^2}_{\text{два слоя } 3 \times 3} = 18C^2 < 25C^2 \text{ (один слой } 5 \times 5)$$

Три свёртки 3×3 (RF = 7×7)

$$params = 9C_{in}C + 9C^2 + 9C C_{out} = 9C(C_{in} + C + C_{out})$$

При $C_{in} = C_{out} = C$:

$$\underbrace{9C^2 + 9C^2 + 9C^2}_{\text{три слоя } 3 \times 3} = 27C^2 < 49C^2 \text{ (один слой } 7 \times 7)$$

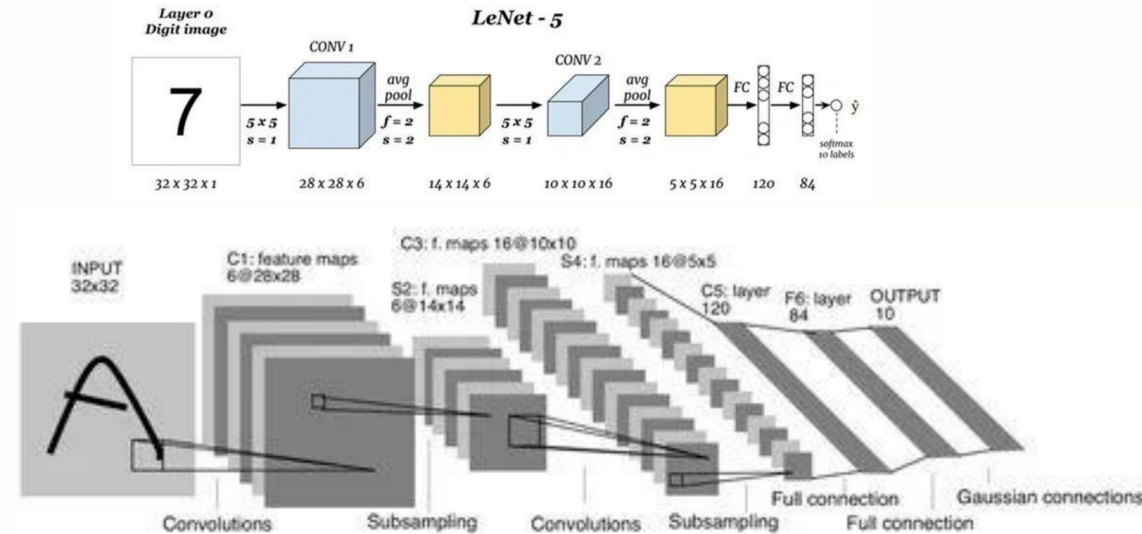
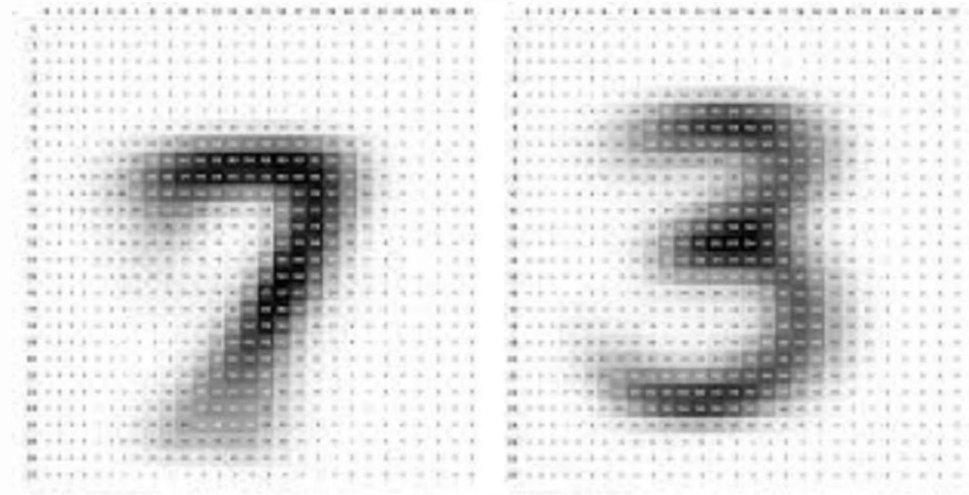
Эквивалент RF	Один большой слой	Стек 3×3	Экономия
5×5	$25C^2$	$18C^2$	~28%
7×7	$49C^2$	$27C^2$	~45%

Чем глубже стек, тем заметнее выигрыш: три слоя 3×3×3 дают почти вдвое меньше параметров, чем одно ядро 7×7×7, при том же рецептивном поле.

3. Эволюция архитектур CNN

История CNN — последовательное решение конкретных технических проблем: вычислительных ограничений, деградации градиента, избыточного числа параметров. Каждая эпохальная архитектура отвечает на вопрос, который предыдущая поставила, но не решила.

LeNet-5 (1990) — Истоки



Первая практическая CNN

Ян ЛеКун разработал LeNet-5 для распознавания рукописных цифр (MNIST, почтовые индексы). Архитектура из 7 слоёв впервые объединила три ключевых элемента: свёрточные слои, субдискретизацию (пулинг) и полносвязный классификатор на выходе.

Структура сети

- Conv(5×5) → Subsampling → Conv(5×5) → Subsampling → 3 полносвязных слоя
- Активация: **tanh / sigmoid** (ReLU ещё не применялась)
- ~60 000 обучаемых параметров

Историческое значение

LeNet доказал принципиальную применимость CNN к реальным задачам распознавания образов. Однако вычислительные ограничения эпохи не позволяли масштабировать идею на ImageNet-подобные задачи ещё два десятилетия.

AlexNet (2012) — Революция глубокого обучения

Прорыв на ImageNet

Крижевский, Сутскевер и Хинтон снизили Top-5 ошибку на ImageNet с ~26% до ~15% — разрыв, превысивший все предшествующие годовые улучшения. Успех стал катализатором современной эры глубокого обучения.

Ключевые инновации AlexNet

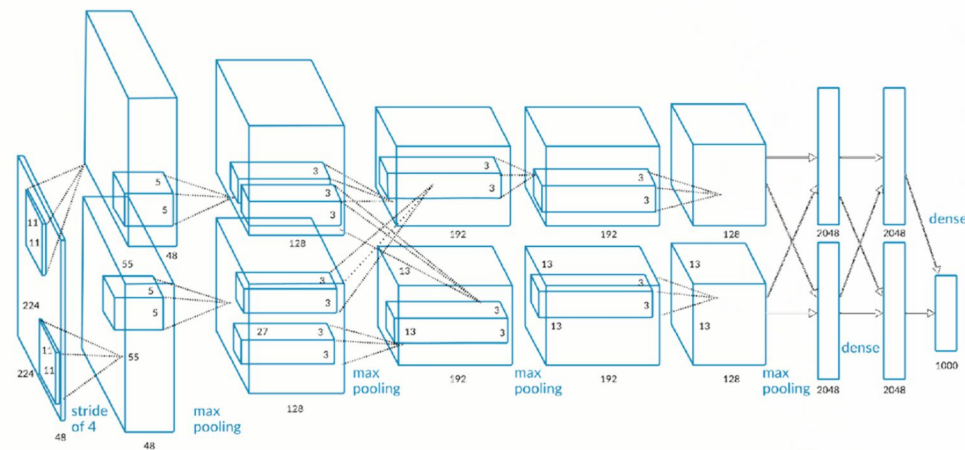
- **ReLU-активации** вместо sigmoid/tanh — устранение проблемы насыщения и ускорение сходимости.
- **GPU-параллелизм**: сеть разбита на два потока для двух GPU (GTX 580), что сделало обучение практически возможным.
- **Dropout** ($p=0.5$) в полносвязных слоях для регуляризации.
- **Аугментация данных**: случайные кропы, отражения, изменение цветового пространства.
- **Local Response Normalization (LRN)** — ранний аналог батч-нормализации.

Архитектурные параметры

8 обучаемых слоёв: 5 свёрточных + 3 полносвязных. Первый свёрточный слой — ядро 11×11 со страйдом 4, что является крупнейшим ядром в истории больших CNN. Общее число параметров ~60M.

Проблема, которую AlexNet не решил

Разнородность архитектуры (смесь крупных и мелких ядер) затрудняла систематическое масштабирование. Этот вопрос поставила и ответила VGG.



VGG-16 (2014) — Сила однородности

Группа Visual Geometry Group (Симонян, Зиссерман) показала: если фиксировать ядро 3×3 и наращивать глубину, точность монотонно растёт. Архитектурная идея — **однородность через повторение блоков**.

Архитектурный принцип

Только ядра 3×3 во всех свёрточных слоях. Глубина достигается повторением блоков «Conv-Conv-Pool». VGG-16: 13 свёрточных + 3 полносвязных = 16 обучаемых слоёв.

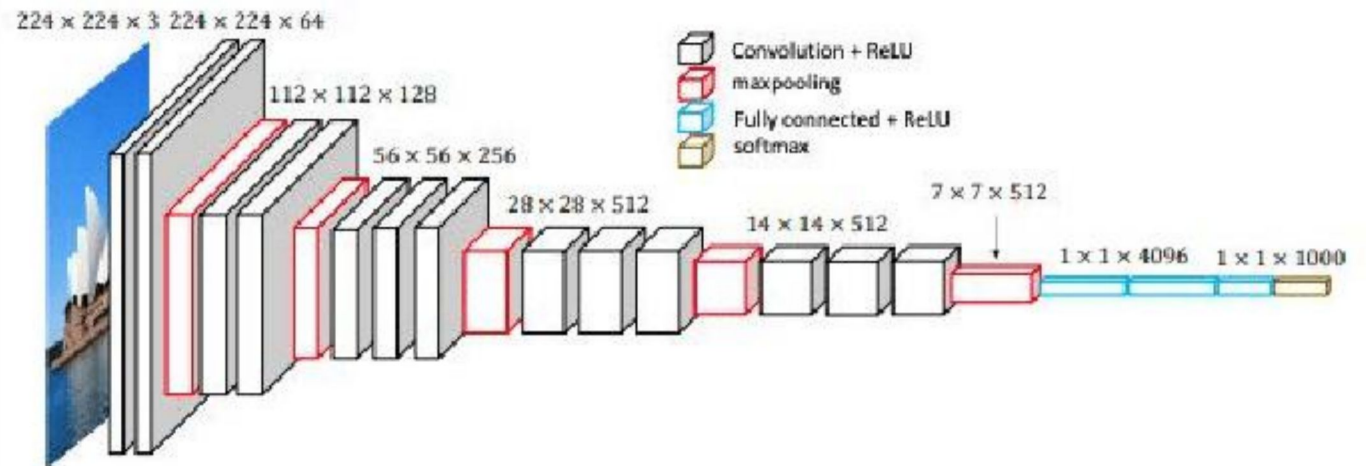
Число параметров

~138 миллионов параметров. Основной «вес» несут три полносвязных слоя (~102M) — именно они стали причиной отказа от FC-слоёв в последующих архитектурах.

Доказанное превосходство малых ядер

Три слоя 3×3 дают $RF = 7 \times 7$ при $3 \cdot 9C^2 = 27C^2$ параметрах против $49C^2$ у одной свёртки 7×7 — экономия 45%.

- ❏ VGG остаётся стандартным baseline для переноса признаков (transfer learning) в задачах компьютерного зрения — несмотря на появление более лёгких архитектур.



ResNet (2015) — Решение проблемы деградации

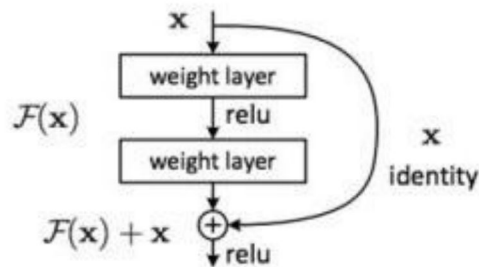
Проблема: деградация при большой глубине

Хэ и соавторы (Microsoft Research) обнаружили парадокс: добавление слоёв к уже хорошо обученной сети **ухудшает** точность на обучающей выборке. Причина — не переобучение, а затухание градиента при обратном распространении сквозь длинные цепочки нелинейных слоёв.

Решение: Skip-connections (Residual Blocks)

$$a^{(l+1)} = f\left(z^{(l+1)}\right) + a^{(l-1)}$$

Обходной путь $a^{(l-1)}$ суммируется с выходом блока преобразования. Сеть учится добавлять **остаток (residual)** к тождественному отображению, что значительно упрощает оптимизацию.



Математика градиентного потока

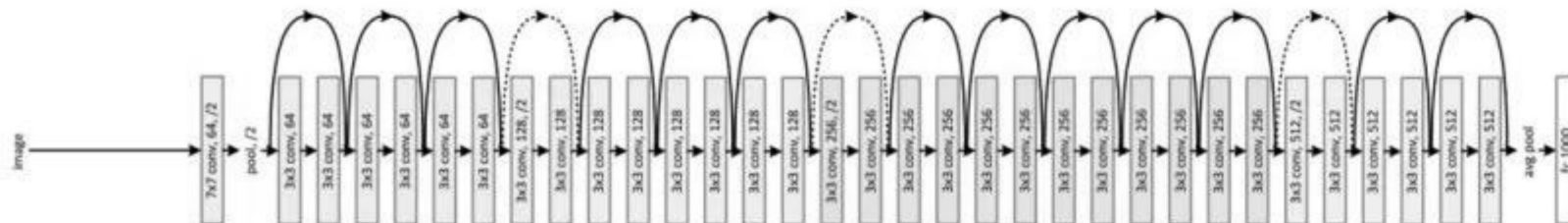
При обратном распространении:

$$\frac{\partial L}{\partial a^{(l-1)}} = \frac{\partial L}{\partial a^{(l+1)}} \left(\frac{\partial f(z^{(l+1)})}{\partial a^{(l-1)}} + I \right)$$

Слагаемое I (единичная матрица от skip-connection) обеспечивает **прямой поток градиента** в ранние слои, минуя цепочку производных активаций. Это предотвращает экспоненциальное затухание даже при глубине 100+ слоёв.

Масштаб результата

ResNet-152 выиграл ILSVRC 2015 с Top-5 ошибкой 3.57% — превзойдя человеческий уровень (~5%). Глубина свыше 1000 слоёв стала технически достижимой.

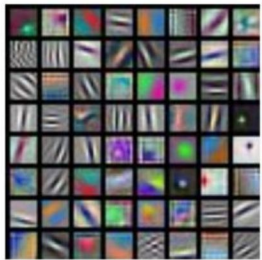


Сравнительная таблица архитектур

Архитектура	Год	Ключевая идея	Глубина	Параметры
LeNet-5	1990	Свёртка + пулинг + FC для распознавания рукописных цифр	7 слоёв	~60K
AlexNet	2012	ReLU + Dropout + GPU + аугментация данных	8 слоёв	~60M
VGG-16	2014	Только ядра 3×3, глубина через повторение блоков	16 слоёв	~138M
ResNet-152	2015	Skip-connections против деградации градиента	152 слоя	~60M

4. Визуализация признаков и интерпретируемость

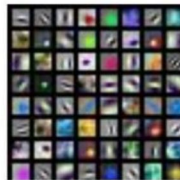
First Layer: Visualize Filters



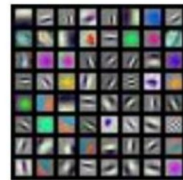
AlexNet:
64 x 3 x 11 x 11



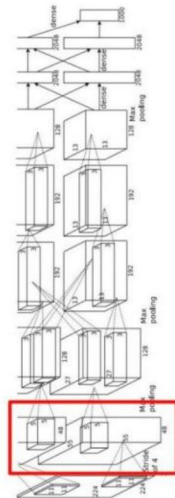
ResNet-18:
64 x 3 x 7 x 7



ResNet-101:
64 x 3 x 7 x 7



DenseNet-121:
64 x 3 x 7 x 7



Krizhevsky, "One weird trick for parallelizing convolutional neural networks", arXiv 2014
He et al., "Deep Residual Learning for Image Recognition", CVPR 2016
Huang et al., "Densely Connected Convolutional Networks", CVPR 2017

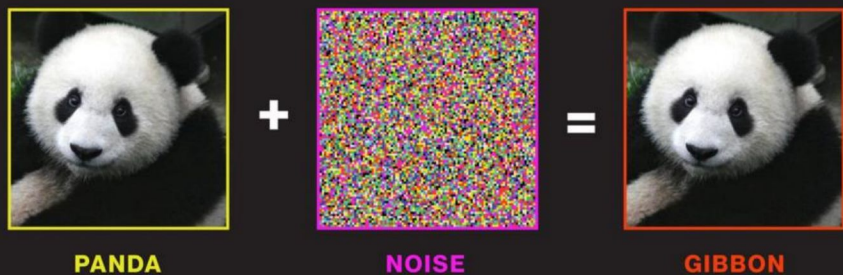
Иерархическая интерпретация

Классическая интуиция, подтверждённая визуализационными исследованиями (Zeiler & Fergus, 2013): признаки CNN организованы иерархически от локальных к глобальным.

- **Слои 1–2:** ориентированные края (аналог клеток Хьюбеля–Визела), цветовые blobs, пространственные частоты.
- **Слои 3–4:** текстуры, решётки, части объектов — угол глаза, колесо, рыба чешуя.
- **Слои 5+:** целые объекты, сцены, семантические категории.

Grad-CAM

Метод Gradient-weighted Class Activation Mapping (Selvaraju et al., 2017) строит тепловую карту релевантности: взвешивает feature maps последнего свёрточного слоя градиентами целевого класса. Визуализирует, **куда «смотрит» сеть** при конкретном предсказании — критически важно для аудита ошибок.



5. Intriguing Properties of Neural Networks

Szegedy et al., 2013 — работа, положившая начало области Adversarial Machine Learning

ARXIV:1312.6199

SZEGEDY · ZAREMBA · SUTSKEVER · GOODFELLOW · 2013

Контекст и мотивация работы

К 2013 году глубокие сети достигли выдающейся точности на стандартных бенчмарках — ImageNet, CIFAR, MNIST. Естественно было предположить, что высокая точность означает **семантически осмысленные внутренние представления** и **устойчивое поведение** вблизи обучающих примеров.

Интуиция «до» работы Szegedy

- Отдельные нейроны кодируют конкретные семантические концепции (гипотеза «бабушкиного нейрона»).
- Модель, правильно классифицирующая изображение, должна правильно классифицировать «похожие» изображения.
- Adversarial perturbations, если и существуют, — редкое исключение для крайних случаев.

Что показала работа

Оба предположения оказались **систематически неверными**. Szegedy et al. сформулировали два контринтуитивных свойства, которые воспроизводятся для любой глубокой сети на любом наборе данных — это не баг конкретной модели, а фундаментальное свойство класса архитектур.

Свойство 1 — Распределённость семантики

Формулировка

Не существует принципиального различия между семантическим содержанием, кодируемым **отдельным нейроном** высокого уровня, и **случайной линейной комбинацией** таких нейронов. Информация содержится во всём пространстве активаций, а не в отдельных «именованных» направлениях базиса.

Экспериментальная проверка

Авторы показали, что оптимизация входа для максимизации случайной проекции $v^T \phi(x)$ на произвольный вектор v в пространстве активаций порождает семантически интерпретируемые изображения так же хорошо, как оптимизация отдельных нейронов.

Следствие для интерпретируемости

Гипотеза о «детекторах концептов» в виде отдельных нейронов — упрощение. Признаки распределены по всему пространству активаций. Это:

- Усложняет механистическую интерпретацию сетей.
- Объясняет, почему adversarial perturbation, задействующая все признаки одновременно, оказывается столь эффективной.
- Предвосхищает современную дискуссию о polysemanticity в нейронах трансформеров.

Свойство 2 — Разрывность отображения вход–выход

Глубокие сети реализуют отображение, являющееся **существенно разрывным** на значительной части входного пространства. Авторы формализовали это через задачу поиска минимального возмущения, меняющего предсказание.

$$r = \arg \min_r \|r\|_2 \quad \text{s.t.} \quad f(x + r) \neq f(x), \quad x + r \in [0, 1]^n$$

Метод решения: L-BFGS

Задача оптимизации решается приближённо через **штрафную формулировку** (box-constrained L-BFGS): минимизируется

$$\|r\|_2 + c \cdot \text{loss}(f(x + r), y_{\text{target}})$$

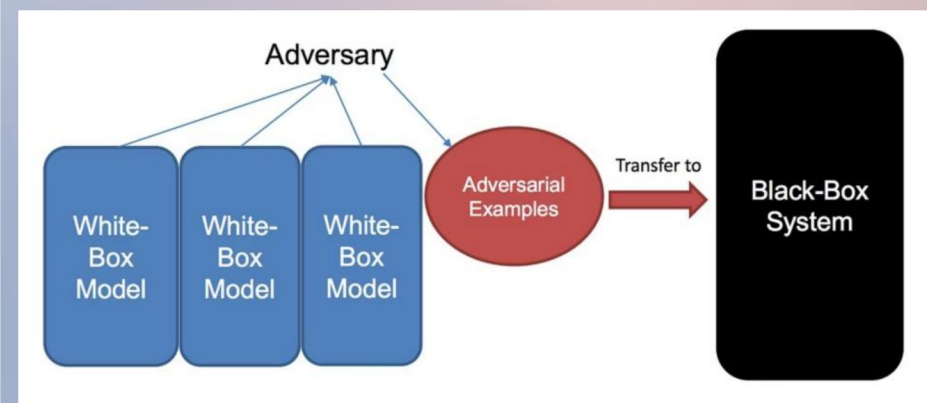
при ограничении на допустимую область пикселей. Итеративный численный метод квазиньютоновского типа.

Ключевые эмпирические результаты

- Найденное r **визуально неразлично** для человека при просмотре изображения.
- Возмущение **надёжно меняет предсказание** для произвольного целевого класса.
- Существуют для всех протестированных архитектур (AlexNet, QuocNet, MNIST-сеть) и наборов данных.

Трансферируемость adversarial примеров

Наиболее практически значимый вывод работы Szegedy et al. — свойство **трансферируемости (transferability)**: возмущение r , построенное для одной модели, способно обмануть другую модель с иной архитектурой или обученную на другом подмножестве данных.



Трансферируемость: значение и следствия

Что наблюдается

Adversarial пример, созданный для модели f_A методом белого ящика (white-box), с ненулевой вероятностью сохраняет adversarial статус при подаче в модель f_B , о внутреннем устройстве которой атакующий ничего не знает.

Теоретическое объяснение

Трансферируемость означает, что adversarial примеры — не артефакт конкретных весов сети, а следствие **системного способа**, которым модели строят решающие границы в высокоразмерном пространстве. Разные модели при обучении на одних данных находят схожие «слабые» направления в пространстве входов.

Практические угрозы

Трансферируемость является **теоретическим фундаментом атак чёрного ящика (transfer-based black-box attacks)**: атакующий обучает суррогатную модель на общедоступных данных, строит adversarial примеры для неё, и атакует целевую систему без доступа к её параметрам. Тема детально разбирается в Лекции 6 курса.

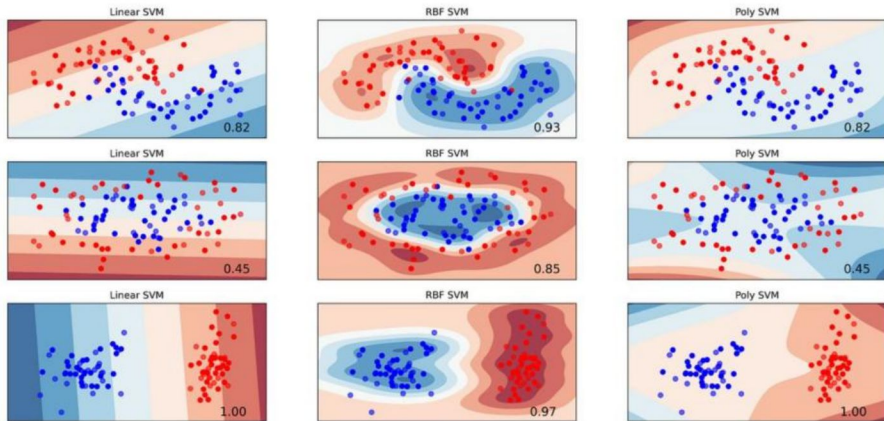
📌 Именно трансферируемость делает adversarial атаки практически угрожающими реальным системам, а не только академическим моделям в условиях белого ящика.

6. Линейная гипотеза Гудфеллоу

Goodfellow, Shlens, Szegedy (2014) — «Explaining and Harnessing Adversarial Examples»

ARXIV:1412.6572

ICLR 2015



Переосмысление природы adversarial примеров

Гипотеза Szegedy (2013): избыточная нелинейность

Оригинальная работа предполагала, что adversarial уязвимость возникает из-за сложной, высоконелинейной природы функции, реализуемой глубокой сетью. Возмущения эксплуатируют «карманы» в нелинейном ландшафте потерь.

Эта интерпретация казалась интуитивно правдоподобной, но имела практический недостаток: не давала простого алгоритма конструирования атак.

Контргипотеза Гудфеллоу: избыточная линейность

Уязвимость возникает **не из-за нелинейности, а из-за линейности** глубоких сетей в высокоразмерном пространстве. В каждой локальной окрестности входного пространства поведение сети достаточно линейно, чтобы простой знак градиента давал эффективное направление атаки.

Это объяснение на порядок проще и порождает вычислительно дешёвый алгоритм — FGSM — требующий лишь одного прохода обратного распространения.

Математический вывод линейной гипотезы

Рассмотрим чисто линейную модель $y = w^T x$ как прокси. При добавлении возмущения η , ограниченного в норме L_∞ :

$$\|\eta\|_\infty \leq \epsilon$$

Изменение выхода модели:

$$w^T(x + \eta) = w^T x + w^T \eta$$

Задача: **максимизировать** $w^T \eta$ при ограничении $\|\eta\|_\infty \leq \epsilon$. Решение достигается выбором знака каждой компоненты возмущения, совпадающим со знаком соответствующего веса:

$$\eta^* = \epsilon \cdot \text{sign}(w)$$

Максимальное достигаемое изменение выхода:

$$w^T \eta^* = \epsilon \cdot \|w\|_1$$

❏ Ключевое наблюдение: хотя каждая отдельная компонента η_i ограничена малой величиной ϵ , суммарный сдвиг $w^T \eta$ растёт как $\epsilon \cdot \|w\|_1 \sim \epsilon \cdot n \cdot \bar{w}$ — линейно с размерностью пространства n . В задачах компьютерного зрения $n \sim 10^4$ – 10^6 , поэтому даже ничтожный ϵ аккумулирует значительное изменение выхода.

Размерность как усилитель атаки

224×224

Пикселей

Стандартный размер входа ImageNet — $224 \times 224 \times 3 \approx 150\text{K}$ значений. Каждый вносит ϵ в суммарный сдвиг.

~150K

Компонент возмущения

Число слагаемых в $\|w\|_1$. При $\epsilon = 0.007$ суммарный сдвиг может превысить 1000 единиц.

0.007

Типичный ϵ

Порог незаметности FGSM в шкале $[0,1]$. Незаметен для человека, но катастрофичен для модели в 224×224 -пространстве.

Именно линейная зависимость суммарного сдвига от размерности объясняет, почему задачи компьютерного зрения (с их десятками – сотнями тысяч пикселей) принципиально уязвимы к L_∞ -ограниченным возмущениям. Это не особенность конкретной архитектуры — это следствие высокой размерности пространства входов.

Fast Gradient Sign Method: вывод

Обобщая линейную интуицию на нелинейную нейронную сеть, Гудфеллоу линеаризует функцию потерь $J(\theta, x, y)$ вокруг текущей точки x . В линейном приближении оптимальное возмущение в норме L_∞ — знак градиента потерь по входу:

$$\eta = \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

Итоговый adversarial пример:

$$x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

Почему $\text{sign}(\cdot)$, а не градиент?

Операция $\text{sign}(\cdot)$ выбрана не произвольно: она является **точным решением** задачи максимизации линейного приближения прироста потерь при ограничении $\|\eta\|_\infty \leq \epsilon$. Это прямое обобщение вывода для линейной модели: $\eta^* = \epsilon \cdot \text{sign}(w) \rightarrow \eta^* = \epsilon \cdot \text{sign}(\nabla_x J)$.

Вычислительная стоимость

FGSM требует **одного прохода обратного распространения** — той же операции, что и стандартный шаг обучения. Стоимость генерации adversarial примера сопоставима со стоимостью одной итерации SGD. Это делает FGSM практически применимым для аугментации обучающей выборки в режиме реального времени.

Алгоритм FGSM: шаг за шагом



Прямой проход

The diagram consists of four horizontal arrows pointing to the right, stacked vertically. Each arrow contains a step of the FGSM algorithm. The arrows are colored in a gradient from dark brown at the top to light brown at the bottom. The first arrow is the shortest, and each subsequent arrow is longer than the one above it.

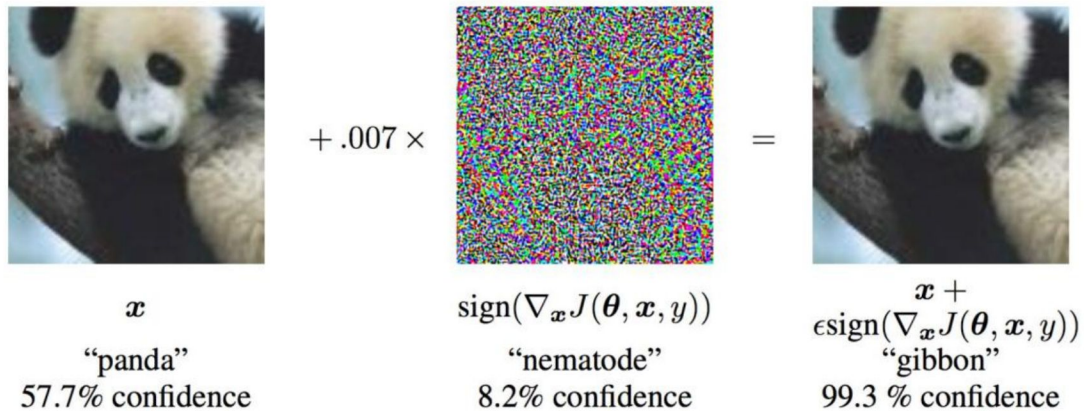
Вычислить loss

Обратный проход

Добавить возмущение

Весь алгоритм выполняется за один прямой и один обратный проход — $O(1)$ относительно числа шагов оптимизации. Единственный гиперпараметр — бюджет ϵ , контролирующий компромисс между незаметностью возмущения и эффективностью атаки.

Визуальный эффект FGSM



Классический пример Гудфеллоу

Исходное изображение панды классифицируется верно с высокой уверенностью. После добавления возмущения $\epsilon \cdot \text{sign}(\nabla_x J)$, полностью незаметного для человека, сеть GoogLeNet предсказывает «гibbon» с уверенностью 99.3%.

Параметры демонстрации

- $\epsilon = 0.007$ – ниже порога заметности для 8-битной палитры
- Возмущение выглядит как случайный шум
- PSNR исходного и adversarial изображений ~ 49 дБ (практически идентичны)

Что это означает?

Решающая граница ImageNet-классификатора проходит настолько близко к обучающим примерам, что линейного шага в направлении роста потерь достаточно для пересечения этой границы.

Геометрически — граница «тонкая» и направлена вдоль почти всех L_∞ -направлений.

FGSM как оптимизационная задача

Важно понимать FGSM в более широком оптимизационном контексте. Атака решает приближённую задачу:

$$\max_{\|\eta\|_{\infty} \leq \epsilon} J(\theta, x + \eta, y) \approx \max_{\|\eta\|_{\infty} \leq \epsilon} [J(\theta, x, y) + \nabla_x J(\theta, x, y)^T \eta]$$

Линеаризация сводит задачу к поиску максимума линейной формы на L_{∞} -шаре, решение которого $\eta^* = \epsilon \cdot \text{sign}(\nabla_x J)$ аналитически точно. Проблема: линеаризация справедлива только **в малой окрестности** точки x .

Точность линеаризации

Для сильно нелинейных участков решающей поверхности одношаговый градиентный сдвиг может не найти точку пересечения решающей границы внутри ϵ -шара. Кривизна (hessian) функции потерь игнорируется.

Локальность знака градиента

Знак градиента указывает направление максимального роста потерь только **локально**. При большом ϵ реальная функция потерь может убывать вдоль этого направления из-за нелинейности.

Attack Success Rate

FGSM при одинаковом бюджете ϵ значительно уступает итеративным методам (PGD, C&W). Это несовершенство мотивирует переход к многошаговым атакам следующей лекции.

Adversarial Training через FGSM

Гудфеллоу с соавторами предложили использовать FGSM непосредственно в обучающем цикле. Модифицированная функция потерь:

$$\tilde{J}(\theta, x, y) = \alpha J(\theta, x, y) + (1 - \alpha) J(\theta, x + \epsilon \operatorname{sign}(\nabla_x J(\theta, x, y)), y)$$

Интерпретация формулы

Параметр $\alpha \in [0, 1]$ взвешивает вклад потерь на исходных (α) и на adversarial примерах ($1 - \alpha$). При $\alpha \rightarrow 0$ обучение ведётся только на атакованных примерах; при $\alpha = 0.5$ — поровну. Гудфеллоу рекомендует $\alpha = 0.5$.

На каждой итерации SGD adversarial пример генерируется «на лету» одним дополнительным проходом backward — стоимость удваивается, но не более.

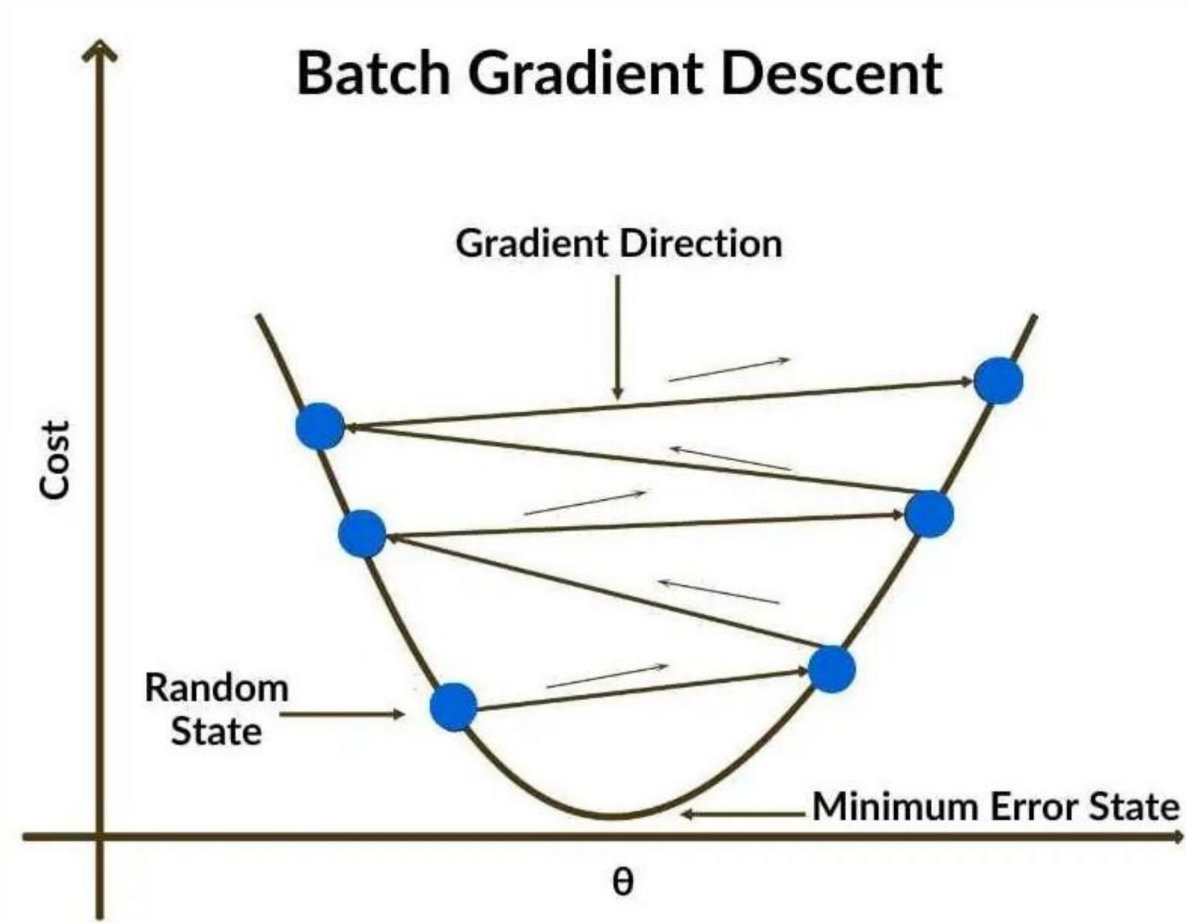
Историческое значение

Это **первая формулировка adversarial training** в истории ML — концепции, которая стала главным методом защиты глубоких моделей. Слабость: FGSM-adversarial training устойчив только к одношаговым атакам; против итеративных (PGD) эффективность падает.

Мы это еще встретим..

В одной из последних лекций adversarial training будем рассматривать в PGD-формулировке Мадри (2018) как минимаксная оптимизация, дающая теоретически обоснованную устойчивость к L_∞ -ограниченным атакам произвольной силы.

7. Ограничения FGSM и мотивация итеративных атак



Где линейная аппроксимация ломается

Нелинейность решающей поверхности

Линеаризация функции потерь $J(\theta, x + \eta, y)$ точна только в малой окрестности точки x . Для сильно нелинейных CNN (особенно при больших ϵ) реальный ландшафт потерь существенно отклоняется от линейного приближения. Одношаговый FGSM может промахнуться мимо решающей границы, даже если та находится в пределах ϵ -шара.

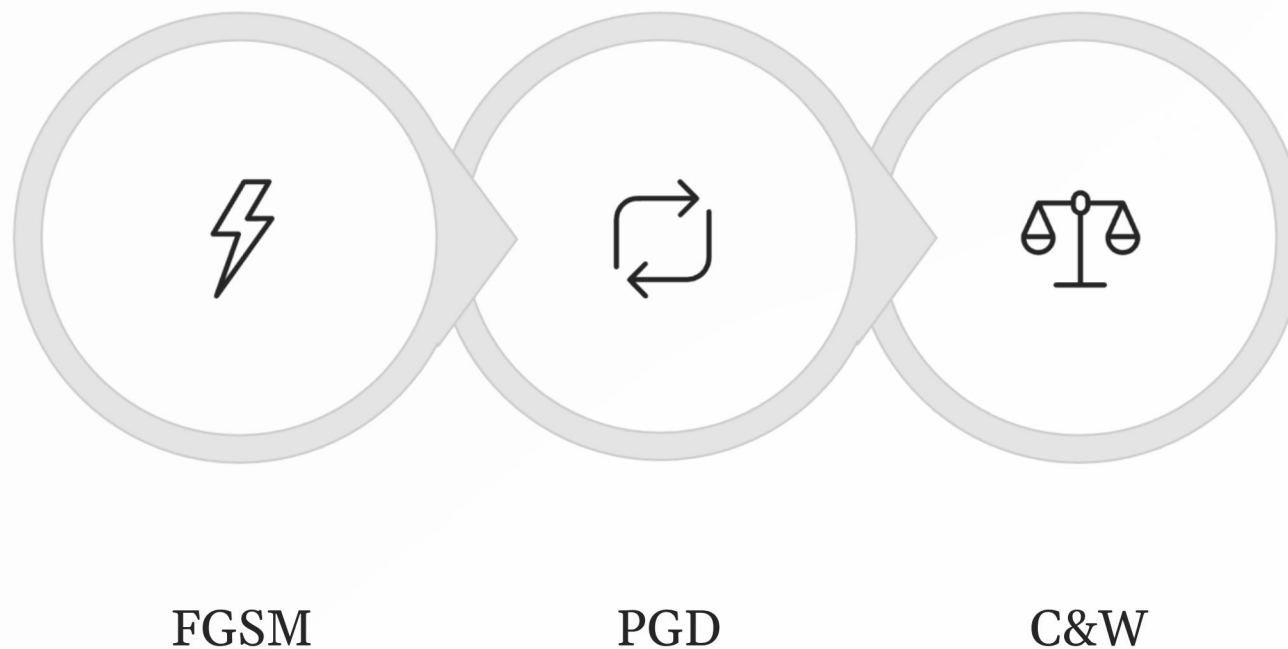
Игнорирование кривизны

FGSM не использует информацию о гессиане функции потерь по входу $\nabla_x^2 J$. В регионах с высокой кривизной направление антиградиента значительно отличается от оптимального направления атаки. Методы второго порядка (Newton-based) нашли бы более эффективное возмущение, но вычислительно слишком дороги для прямого применения.

Практический разрыв в Attack Success Rate

При фиксированном бюджете ϵ итеративный PGD (Projected Gradient Descent, Madry et al. 2018) с 20–40 шагами достигает Attack Success Rate на 30–60 процентных пунктов выше, чем FGSM. Это несовершенство FGSM превращает его из «полноценной атаки» в «быстрый нижний предел силы защиты».

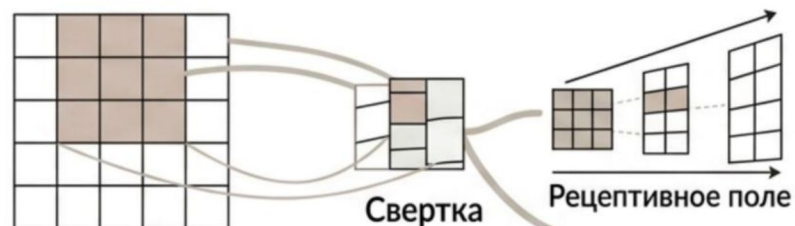
От FGSM к итеративным атакам



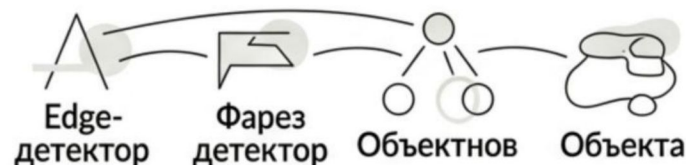
Каждый следующий метод точнее решает задачу $\max_{\|\eta\| \leq \epsilon} J(\theta, x + \eta, y)$ за счёт большего числа итераций или более гибкой формулировки.

CNN → Adversarial ML

МАТЕМАТИКА CNN



ОПЕРАЦИЯ СВЕРТКИ: выделение локальных признаков, общие веса.



ИЕРАРХИЯ ПРИЗНАКОВ: от простых краев к сложным объектам.

ИЕРАРХИЯ ПРИЗНАКОВ: от ResNet- блок  Обходное соединение

СОСТЯЗАТЕЛЬНАЯ УЯЗВИМОСТЬ



ПРЕРЫВИСТОЕ ОТОБРАЖЕНИЕ: ГИББОН
малые изменения -> большие скачки в выходе.



ПЕРЕНОСИМОСТЬ: атаки работают на разных моделях.

Понимание архитектуры CNN не просто академический бэкграунд: именно высокая размерность входного пространства, на работу с которым оптимизированы CNN, создаёт структурные условия для существования adversarial examples согласно линейной гипотезе Гудфеллоу.

Ключевые формулы лекции

Математика CNN

$$Z_{i,j} = \sum_{c,m,n} X_{i+m,j+n,c} \cdot K_{m,n,c} + b$$

$$H_{out} = \left\lfloor \frac{H - k + 2p}{s} \right\rfloor + 1$$

$$RF_L = RF_{L-1} + (k_L - 1) \prod_{i=1}^{L-1} s_i$$

$$a^{(l+1)} = f(z^{(l+1)}) + a^{(l-1)}$$

Adversarial ML

$$r = \arg \min_r \|r\|_2 \text{ s.t. } f(x + r) \neq f(x)$$

$$\eta^* = \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

$$x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

$$\tilde{J} = \alpha J(\theta, x, y) + (1 - \alpha) J(\theta, x_{adv}, y)$$

Итоги лекции и дальнейший путь

Математика свёртки

Формализована операция 2D-свёртки, вывод размера feature map при произвольных страйде и паддинге, рекуррентная формула рецептивного поля.

Эволюция архитектур

LeNet → AlexNet → VGG → ResNet: каждая эпоха решает конкретную проблему. Skip-connections — математически элегантное решение деградации градиента.

Intriguing properties

Семантика распределена в пространстве активаций. Отображение вход-выход существенно разрывно. Adversarial примеры трансформируются между моделями.

Линейная гипотеза и FGSM

Уязвимость обусловлена линейностью в высокоразмерном пространстве. FGSM — точное решение линеаризованной задачи максимизации потерь в L_∞ .

Следующая лекция

Итеративные evasion-атаки: **PGD** (Projected Gradient Descent) и **Carlini-Wagner**. Почему одношаговые атаки недостаточны, как проекция на ϵ -шар обеспечивает ограничение норм при многошаговой оптимизации, и как оценивать реальную устойчивость защит.